Document number: P0935R0
Date: 2018-02-12
Reply-to: Tim Song <rs2740 at gmail dot com>
Audience: LWG

Eradicating unnecessarily explicit default constructors from the standard library

Discussion:

Explicit default constructors block copy-list-initialization from `{}`. This typically happens:

- when a `T` object is initialized with `= {};`
- when a function returning a `T` returns `{};`
- when a function taking a `T` is given `{}` as the argument;
- as a special case of the above, when a `T` object is assigned `{};`
- when no explicit initializer is supplied for a `T` element of an aggregate (and no default member initializer exists) (see 11.6.1 [dcl.init.aggr] p5).

In other words, if `T` is a class type with an explicit default constructor, all of the following are ill-formed:

```
struct itpair { int i; T t; } x = { 0 };
itpair y{};    // not even using direct-list-initialization on the aggregate helps
itpair z = {};
T t = {};
t = {};        // assuming that T has a copy/move assignment operator that is selected by overload resolution
```

For most `T`s, this is quite surprising behavior and highly unlikely to be desirable. (There is currently an open Ranges TS issue asking whether `DefaultConstructible` should permit such types.) To take an example from the wording below, it seems reasonable to store a random number distribution as a member of an aggregate and implicitly initialize it if the default argument values are suitable, but we can't do this currently:

```
struct normrd {
    std::mt19937 engine;
    std::normal_distribution<> dist;
};
normrd standard_normal_dist = { make_seeded<std::mt19937>() }; // error
```

Moreover, if the type at issue is a C++03 type, then this also presents a compatibility problem because in C++03 an aggregate member without an explicit initializer is value-initialized, which tolerates explicit default constructors just fine. For example, with the container adaptors:

```
struct tagged_queue {
    std::string tag;
    std::queue<int> queue;
};
tagged_queue q = { "queue1" }; // OK in C++03; error now
```

With the exception of the explicit default constructors intentionally introduced by LWG 2510, most explicit default constructors in the library appear to arise out of historical accident rather than intentional design: they are constructors taking one or more parameters, all of which have default arguments, and so the `explicit` was added to prevent defining an undesirable implicit conversion from the type of the first parameter. At the time of their design, `explicit` simply had no effect in the other cases. It is also worth noting that the vast majority of them (container adaptors, random number engines, certain random number distributions, string streams) all have analogs with non-explicit default constructors (containers, random number engine adaptors, sampling distributions, file streams).

We should not use explicit default constructors without a good reason, at least for classes that can conceivably be used in a copy-list-initialization context. LWG 2193 eliminated a number of such constructors for types with initializer-list constructors. The proposed wording below finishes the job.

Note that this paper intentionally does not cover or propose changes on the explicitness of constructors taking >1 parameters. That more difficult question is covered by LWG 2307.

Proposed wording:

This wording is relative to N4713.

> [*Drafting note*: All facets have explicit default constructors, but they also have protected destructors and so can't normally be constructed in one of the contexts noted above. Even ignoring the access issue, a default-constructed facet

is meant to be `deleted` by the locale, and `new` is always direct-initialization. Thus, no change is proposed for them. — *end drafting note*]

[*Drafting note*: `scoped_lock<>` has an explicit default constructor in the empty pack case, but I can't think of a reason why one would use it in non-generic code, and in generic code direct-initialization is required anyway. Thus no change to it is proposed. — *end drafting note*]

1. Edit 26.6.4.1 [queue.defn], class template `queue` synopsis, as indicated:

```
  […]

  public:
    queue() : queue(Container()) {}
    explicit queue(const Container&);
    explicit queue(Container&& = Container());
  […]
```

2. Edit 26.6.4.2 [queue.cons] before p2 as indicated:

```
explicit queue(Container&& cont = Container());
```

-2- *Effects:* Initializes `c` with `std::move(cont)`.

3. Edit 26.6.5 [priority.queue], class template `priority_queue` synopsis, as indicated:

[*Drafting note*: This contains a drive-by fix that makes the two-parameter rvalue-`Container` constructor non-explicit, to match the two-parameter const lvalue-`Container` constructor. Regardless of what one might think about the merits of `explicit` on the two-parameter form, it seems nonsensical for it to turn on the value category of one of the arguments. — *end drafting note*]

```
  […]

  public:
    priority_queue() : priority_queue(Compare()) {}
    explicit priority_queue(const Compare& x) : priority_queue(x, Container()) {}
    priority_queue(const Compare& x, const Container&);
    explicit priority_queue(const Compare& x = Compare(), Container&&= Container());
  […]
```

4. Edit 26.6.5.1 [priqueue.cons] before p1 as indicated:

```
priority_queue(const Compare& x, const Container& y);
explicit priority_queue(const Compare& x = Compare(), Container&& y = Container());
```

-1- *Requires:* […]

-2- *Effects:* […]

5. Edit 26.6.6.1 [stack.defn], class template `stack` synopsis, as indicated:

```
  […]

  public:
    stack() : stack(Container()) {}
    explicit stack(const Container&);
    explicit stack(Container&& = Container());
  […]
```

6. Edit 26.6.6.2 [stack.cons] before p2 as indicated:

```
explicit stack(Container&& cont = Container());
```

-2- *Effects:* Initializes `c` with `std::move(cont)`.

7. Edit 29.6.3.1 [rand.eng.lcong] after p1, class template `linear_congruential_engine` synopsis, as indicated:

```
  […]

    // constructors and seeding functions
    linear_congruential_engine() : linear_congruential_engine(default_seed) {}
    explicit linear_congruential_engine(result_type s = default_seed);
  […]
```

8. Edit 29.6.3.1 [rand.eng.lcong] before p5 as indicated:

```
explicit linear_congruential_engine(result_type s~~ = default_seed~~);
```

-5- *Effects:* […]

9. Edit 29.6.3.2 [rand.eng.mers] after p3, class template `mersenne_twister_engine` synopsis, as indicated:

```
  […]

    // constructors and seeding functions
    mersenne_twister_engine() : mersenne_twister_engine(default_seed) {}
    explicit mersenne_twister_engine(result_type s~~ = default_seed~~);
  […]
```

10. Edit 29.6.3.2 [rand.eng.mers] before p6 as indicated:

```
explicit mersenne_twister_engine(result_type s~~ = default_seed~~);
```

-6- *Effects:* […]

-7- *Complexity:* […]

11. Edit 29.6.3.3 [rand.eng.sub] after p4, class template `subtract_with_carry_engine` synopsis, as indicated:

```
  […]

    // constructors and seeding functions
    subtract_with_carry_engine() : subtract_with_carry_engine(default_seed) {}
    explicit subtract_with_carry_engine(result_type s~~ = default_seed~~);
  […]
```

12. Edit 29.6.3.3 [rand.eng.sub] before p7 as indicated:

```
explicit subtract_with_carry_engine(result_type s~~ = default_seed~~);
```

-7- *Effects:* […]

-8- *Complexity:* […]

13. Edit 29.6.6 [rand.device] after p2, class `random_device` synopsis, as indicated:

```
  […]

  // constructors
  random_device() : random_device(implementation-defined) {}
  explicit random_device(const string& token~~ = implementation-defined~~);

  […]
```

14. Edit 29.6.6 [rand.device] p3 as indicated:

```
random_device() : random_device(implementation-defined) {}
explicit random_device(const string& token~~ = implementation-defined~~);
```

-3- *Effects:* Constructs a `random_device` nondeterministic uniform random bit generator object. The semantics~~ and default value~~ of the `token` parameter and the token value used by the default constructor are implementation-defined. [*Footnote:*…]

-4- *Throws:* […]

15. Edit 29.6.8.2.1 [rand.dist.uni.int] after p1, class template `uniform_int_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    uniform_int_distribution() : uniform_int_distribution(0) {}
    explicit uniform_int_distribution(IntType a~~ = 0~~, IntType b = numeric_limits<IntType>::max());
  […]
```

16. Edit 29.6.8.2.1 [rand.dist.uni.int] before p2 as indicated:

```
explicit uniform_int_distribution(IntType a~~ = 0~~, IntType b = numeric_limits<IntType>::max());
```

-2- *Requires:* […]

-3- *Effects:* […]

17. Edit 29.6.8.2.2 [rand.dist.uni.real] after p1, class template `uniform_real_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    uniform_real_distribution() : uniform_real_distribution(0.0) {}
    explicit uniform_real_distribution(RealType a = 0.0, RealType b = 1.0);
  […]
```

18. Edit 29.6.8.2.2 [rand.dist.uni.real] before p2 as indicated:

```
explicit uniform_real_distribution(RealType a = 0.0, RealType b = 1.0);
```

-2- *Requires:* […]

-3- *Effects:* […]

19. Edit 29.6.8.3.1 [rand.dist.bern.bernoulli] after p1, class `bernoulli_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    bernoulli_distribution() : bernoulli_distribution(0.5) {}
    explicit bernoulli_distribution(double p = 0.5);
  […]
```

20. Edit 29.6.8.3.1 [rand.dist.bern.bernoulli] before p2 as indicated:

```
explicit bernoulli_distribution(double p = 0.5);
```

-2- *Requires:* […]

-3- *Effects:* […]

21. Edit 29.6.8.3.2 [rand.dist.bern.bin] after p1, class template `binomial_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    binomial_distribution() : binomial_distribution(1) {}
    explicit binomial_distribution(IntType t = 1, double p = 0.5);
  […]
```

22. Edit 29.6.8.3.2 [rand.dist.bern.bin] before p2 as indicated:

```
explicit binomial_distribution(IntType t = 1, double p = 0.5);
```

-2- *Requires:* […]

-3- *Effects:* […]

23. Edit 29.6.8.3.3 [rand.dist.bern.geo] after p1, class template `geometric_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    geometric_distribution() : geometric_distribution(0.5) {}
    explicit geometric_distribution(double p = 0.5);
  […]
```

24. Edit 29.6.8.3.3 [rand.dist.bern.geo] before p2 as indicated:

```
explicit geometric_distribution(double p = 0.5);
```

-2- *Requires:* […]

-3- *Effects:* […]

25. Edit 29.6.8.3.4 [rand.dist.bern.negbin] after p1, class template `negative_binomial_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    negative_binomial_distribution() : negative_binomial_distribution(1) {}
    explicit negative_binomial_distribution(IntType k = 1, double p = 0.5);
  […]
```

26. Edit 29.6.8.3.4 [rand.dist.bern.negbin] before p2 as indicated:

```
explicit negative_binomial_distribution(IntType k = 1, double p = 0.5);
```

-2- *Requires:* […]

-3- *Effects:* […]

27. Edit 29.6.8.4.1 [rand.dist.pois.poisson] after p1, class template `poisson_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    poisson_distribution() : poisson_distribution(1.0) {}
    explicit poisson_distribution(double mean = 1.0);
  […]
```

28. Edit 29.6.8.4.1 [rand.dist.pois.poisson] before p2 as indicated:

```
explicit poisson_distribution(double mean = 1.0);
```

-2- *Requires:* […]

-3- *Effects:* […]

29. Edit 29.6.8.4.2 [rand.dist.pois.exp] after p1, class template `exponential_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    exponential_distribution() : exponential_distribution(1.0) {}
    explicit exponential_distribution(RealType lambda = 1.0);
  […]
```

30. Edit 29.6.8.4.2 [rand.dist.pois.exp] before p2 as indicated:

```
explicit exponential_distribution(RealType lambda = 1.0);
```

-2- *Requires:* […]

-3- *Effects:* […]

31. Edit 29.6.8.4.3 [rand.dist.pois.gamma] after p1, class template `gamma_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    gamma_distribution() : gamma_distribution(1.0) {}
    explicit gamma_distribution(RealType alpha = 1.0, RealType beta = 1.0);
  […]
```

32. Edit 29.6.8.4.3 [rand.dist.pois.gamma] before p2 as indicated:

```
explicit gamma_distribution(RealType alpha = 1.0, RealType beta = 1.0);
```

-2- *Requires:* […]

-3- *Effects:* […]

33. Edit 29.6.8.4.4 [rand.dist.pois.weibull] after p1, class template `weibull_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    weibull_distribution() : weibull_distribution(1.0) {}
    explicit weibull_distribution(RealType a = 1.0, RealType b = 1.0);
  […]
```

34. Edit 29.6.8.4.4 [rand.dist.pois.weibull] before p2 as indicated:

```
explicit weibull_distribution(RealType a~~ = 1.0~~, RealType b = 1.0);
```

-2- *Requires:* […]

-3- *Effects:* […]

35. Edit 29.6.8.4.5 [rand.dist.pois.extreme] after p1, class template `extreme_value_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    extreme_value_distribution() : extreme_value_distribution(0.0) {}
    explicit extreme_value_distribution(RealType a~~ = 0.0~~, RealType b = 1.0);
  […]
```

36. Edit 29.6.8.4.5 [rand.dist.pois.extreme] before p2 as indicated:

```
explicit extreme_value_distribution(RealType a~~ = 0.0~~, RealType b = 1.0);
```

-2- *Requires:* […]

-3- *Effects:* […]

37. Edit 29.6.8.5.1 [rand.dist.norm.normal] after p1, class template `normal_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    normal_distribution() : normal_distribution(0.0) {}
    explicit normal_distribution(RealType mean~~ = 0.0~~, RealType stddev = 1.0);
  […]
```

38. Edit 29.6.8.5.1 [rand.dist.norm.normal] before p2 as indicated:

```
explicit normal_distribution(RealType mean~~ = 0.0~~, RealType stddev = 1.0);
```

-2- *Requires:* […]

-3- *Effects:* […]

39. Edit 29.6.8.5.2 [rand.dist.norm.lognormal] after p1, class template `lognormal_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    lognormal_distribution() : lognormal_distribution(0.0) {}
    explicit lognormal_distribution(RealType m~~ = 0.0~~, RealType s = 1.0);
  […]
```

40. Edit 29.6.8.5.2 [rand.dist.norm.lognormal] before p2 as indicated:

```
explicit lognormal_distribution(RealType m~~ = 0.0~~, RealType s = 1.0);
```

-2- *Requires:* […]

-3- *Effects:* […]

41. Edit 29.6.8.5.3 [rand.dist.norm.chisq] after p1, class template `chi_squared_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    chi_squared_distribution() : chi_squared_distribution(1) {}
    explicit chi_squared_distribution(RealType n~~ = 1~~);
  […]
```

42. Edit 29.6.8.5.3 [rand.dist.norm.chisq] before p2 as indicated:

```
explicit chi_squared_distribution(RealType n~~ = 1~~);
```

-2- *Requires:* […]

-3- *Effects:* […]

43. Edit 29.6.8.5.4 [rand.dist.norm.cauchy] after p1, class template `cauchy_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    cauchy_distribution() : cauchy_distribution(0.0) {}
    explicit cauchy_distribution(RealType a = 0.0, RealType b = 1.0);
  […]
```

44. Edit 29.6.8.5.4 [rand.dist.norm.cauchy] before p2 as indicated:

```
explicit cauchy_distribution(RealType a = 0.0, RealType b = 1.0);
```

-2- *Requires:* […]

-3- *Effects:* […]

45. Edit 29.6.8.5.5 [rand.dist.norm.f] after p1, class template `fisher_f_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    fisher_f_distribution() : fisher_f_distribution(1) {}
    explicit fisher_f_distribution(RealType m = 1, RealType n = 1);
  […]
```

46. Edit 29.6.8.5.5 [rand.dist.norm.f] before p2 as indicated:

```
explicit fisher_f_distribution(RealType m = 1, RealType n = 1);
```

-2- *Requires:* […]

-3- *Effects:* […]

47. Edit 29.6.8.5.6 [rand.dist.norm.t] after p1, class template `student_t_distribution` synopsis, as indicated:

```
  […]

    // constructors and reset functions
    student_t_distribution() : student_t_distribution(1) {}
    explicit student_t_distribution(RealType n = 1);
  […]
```

48. Edit 29.6.8.5.6 [rand.dist.norm.t] before p2 as indicated:

```
explicit student_t_distribution(RealType n = 1);
```

-2- *Requires:* […]

-3- *Effects:* […]

49. Edit 30.8.2 [stringbuf], class template `basic_stringbuf` synopsis, as indicated:

```
  […]

    // 30.8.2.1 [stringbuf.cons], constructors
    basic_stringbuf() : basic_stringbuf(ios_base::in | ios_base::out) {}
    explicit basic_stringbuf(
      ios_base::openmode which = ios_base::in | ios_base::out);
  […]
```

50. Edit 30.8.2.1 [stringbuf.cons] before p1 as indicated:

```
explicit basic_stringbuf(
  ios_base::openmode which = ios_base::in | ios_base::out);
```

-1- *Effects:* […]

-2- *Postconditions:* […]

51. Edit 30.8.3 [istringstream], class template `basic_istringstream` synopsis, as indicated:

```
  […]

    // 30.8.3.1 [istringstream.cons], constructors
    basic_istringstream() : basic_istringstream(ios_base::in) {}
    explicit basic_istringstream(
      ios_base::openmode which = ios_base::in);
  […]
```

52. Edit 30.8.3.1 [istringstream.cons] before p1 as indicated:

```
explicit basic_istringstream(ios_base::openmode which = ios_base::in);
```

-1- *Effects:* […]

53. Edit 30.8.4 [ostringstream], class template `basic_ostringstream` synopsis, as indicated:

```
  […]

    // 30.8.4.1 [ostringstream.cons], constructors
    basic_ostringstream() : basic_ostringstream(ios_base::out) {}
    explicit basic_ostringstream(
      ios_base::openmode which = ios_base::out);
  […]
```

54. Edit 30.8.4.1 [ostringstream.cons] before p1 as indicated:

```
explicit basic_ostringstream(
  ios_base::openmode which = ios_base::out);
```

-1- *Effects:* […]

55. Edit 30.8.5 [stringstream], class template `basic_stringstream` synopsis, as indicated:

```
  […]

    // 30.8.5.1 [stringstream.cons], constructors
    basic_stringstream() : basic_stringstream(ios_base::out | ios_base::in) {}
    explicit basic_stringstream(
      ios_base::openmode which = ios_base::out | ios_base::in);
  […]
```

56. Edit 30.8.5.1 [stringstream.cons] before p1 as indicated:

```
explicit basic_stringstream(
  ios_base::openmode which = ios_base::out | ios_base::in);
```

-1- *Effects:* […]

57. Edit 31.10 [re.results] after p4, class template `match_results` synopsis, as indicated:

```
  […]

    // 31.10.1 [re.results.const], construct/copy/destroy
    match_results() : match_results(Allocator()) {}
    explicit match_results(const Allocator& a = Allocator());
  […]
```

58. Edit 31.10.1 [re.results.const] before p1 as indicated:

[*Drafting note*: This contains a drive-by editorial fix for a missing `explicit` in the constructor description. — *end drafting note*]

```
explicit match_results(const Allocator&amp; a = Allocator());
```

-1- *Effects:* […]

-2- *Postconditions:* […]

59. Edit D.7.1 [depr.strstreambuf], class `strstreambuf` synopsis, as indicated:

```
  […]

  strstreambuf() : strstreambuf(0) {}
```

```
    explicit strstreambuf(streamsize alsize_arg = 0);
  […]
```

60. Edit D.7.1.1 [depr.strstreambuf.cons] before p1 as indicated:

```
explicit strstreambuf(streamsize alsize_arg = 0);
```

-1- *Effects:* […]

61. Edit D.18.1 [depr.conversions.string] after p1, class template `wstring_convert` synopsis, as indicated:

```
  […]

      wstring_convert() : wstring_convert(new Codecvt) {}
      explicit wstring_convert(Codecvt* pcvt = new Codecvt);
  […]
```

62. Edit D.18.1 [depr.conversions.string] before p16 as indicated:

```
explicit wstring_convert(Codecvt* pcvt = new Codecvt);
wstring_convert(Codecvt* pcvt, state_type state);
explicit wstring_convert(const byte_string& byte_err,
    const wide_string& wide_err = wide_string());
```

-16- *Requires:* […]

-17- *Effects:* […]

63. Edit D.18.2 [depr.conversions.buffer] after p1, class template `wbuffer_convert` synopsis, as indicated:

```
  […]

      wbuffer_convert() : wbuffer_convert(nullptr) {}
      explicit wbuffer_convert(streambuf* bytebuf = nullptr,
                               Codecvt* pcvt = new Codecvt,
                               state_type state = state_type());
  […]
```

64. Edit D.18.2 [depr.conversions.buffer] before p9 as indicated:

```
explicit wbuffer_convert(
    streambuf* bytebuf = nullptr,
    Codecvt* pcvt = new Codecvt,
    state_type state = state_type());
```

-9- *Requires:* […]

-10- *Effects:* […]